

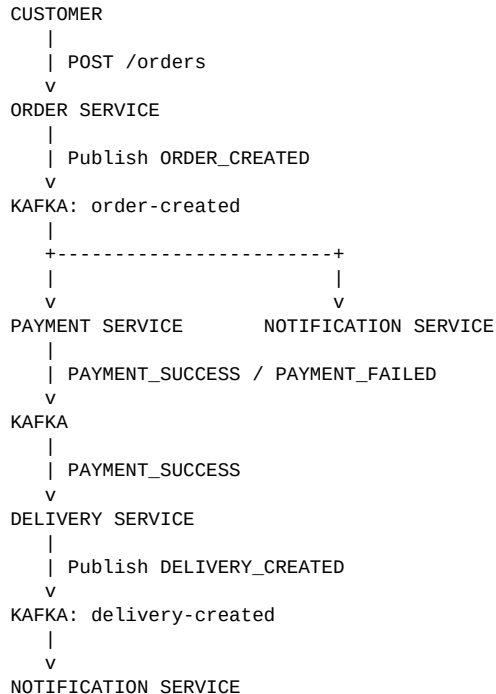
Kafka Microservices Project

Order Processing System — Project Requirements

1. Objective

Build an event-driven Order Processing System using Spring Boot Microservices and Apache Kafka. The system must demonstrate asynchronous communication, Kafka producers/consumers, consumer groups, partitions, offsets, retries, idempotency and Dead Letter Topics (DLT).

2. Overall Business Flow



3. Order Service

Responsibilities: create and store orders, publish ORDER_CREATED, and maintain order status.

POST /orders

Request:

```
{
  "customerId": 101,
  "customerName": "Rahul",
  "productId": 501,
  "productName": "Laptop",
  "quantity": 1,
  "amount": 75000,
  "deliveryAddress": "Bangalore"
}
```

Response:

```
{
  "orderId": 1001,
  "status": "CREATED"
}
```

Topic: order-created

Event:

```
{
  "eventId": "EVT-10001",
  "eventType": "ORDER_CREATED",
  "orderId": 1001,
  "customerId": 101,
  "amount": 75000,
  "deliveryAddress": "Bangalore"
}
```

4. Payment Service

Consume ORDER_CREATED, simulate/process payment, store the result, and publish the payment outcome.

payment-success:

```
{
  "eventId": "PAY-10001",
  "eventType": "PAYMENT_SUCCESS",
  "orderId": 1001,
  "customerId": 101,
  "amount": 75000,
}
```

```

    "paymentId": "TXN-90001",
    "paymentMethod": "UPI",
    "paymentStatus": "SUCCESS"
  }

  payment-failed:
  {
    "eventId": "PAY-10002",
    "eventType": "PAYMENT_FAILED",
    "orderId": 1002,
    "customerId": 102,
    "amount": 45000,
    "paymentId": "TXN-90002",
    "paymentStatus": "FAILED",
    "reason": "INSUFFICIENT_FUNDS"
  }

```

5. Delivery Service

Consume successful payment events, create a delivery, generate a tracking number, store delivery information and publish DELIVERY_CREATED.

```

Topic: delivery-created

{
  "eventId": "DEL-10001",
  "eventType": "DELIVERY_CREATED",
  "orderId": 1001,
  "customerId": 101,
  "trackingNumber": "TRK-50001",
  "deliveryAddress": "Bangalore",
  "deliveryStatus": "CREATED"
}

```

Statuses: CREATED, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERED, CANCELLED.

6. Notification Service

Consume relevant events and simulate SMS/email notifications. No real notification provider is required.

```

Events:
ORDER_CREATED
PAYMENT_SUCCESS
PAYMENT_FAILED
DELIVERY_CREATED
DELIVERY_OUT_FOR_DELIVERY
ORDER_DELIVERED

```

```

Example:
SMS SENT
Customer: 101
Order: 1001
Your payment of Rs.75,000 was successful.

```

7. Kafka Requirements

Minimum topics: order-created, payment-success, payment-failed, delivery-created. Use meaningful consumer groups such as payment-service-group, delivery-service-group and notification-service-group. Understand Topic, Partition, Offset, Producer, Consumer and Consumer Group.

8. Mandatory Kafka Exploration

- Multiple Consumers — demonstrate the same event being consumed by independent consumer groups.
- Partitions — use multiple partitions and orderId as the message key; observe ordering and distribution.
- Consumer Scaling — run multiple Payment Service instances in the same group.
- Failure & Retry — stop Payment Service, create an order, restart it and verify processing.
- Duplicate Message — process the same event twice without duplicate business processing using eventId/idempotency.
- Dead Letter Topic — after retries, send an unprocessable message to payment-success.DLT.

9. Common Event Fields

Where applicable, events should contain eventId, eventType, orderId, customerId and eventTime. Students should decide additional fields required by each service.

10. Final Expected Flow

```

POST /orders
|
v
Order Service
|
| ORDER_CREATED
v
Kafka
|
v
Payment Service
|

```

```
graph TD
    A[PAYMENT_SUCCESS] --> B[Kafka]
    B --> C[Delivery Service]
    C --> D[DELIVERY_CREATED]
    D --> E[Kafka]
    E --> F[Notification Service]
    F --> G[Notification Sent]
```

Failed payment: ORDER_CREATED → Payment Service → PAYMENT_FAILED → Notification Service → Payment Failure Notification.

11. Deliverables

- Source code in GitHub
- Architecture diagram
- Kafka topic and consumer-group details
- Postman collection
- README with setup instructions
- Screenshots showing Kafka topics, messages, partitions and consumer groups
- Demo of successful flow, failure/retry, duplicate event and DLT